IMDDB-RNN,LSTM,GRU PERFOMANCE COMPARISON

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import time

from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU, Dense,
Dropout
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

vocab_size = 10000
max_length = 200

(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

X_train = pad_sequences(X_train, maxlen=max_length)
X_test = pad_sequences(X_test, maxlen=max_length)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

def build_model(model_type):
    model = Sequential([
        Embedding(vocab_size, 64, mask_zero=True)
    ])

    if model_type == "RNN":
        model.add(SimpleRNN(64))
    elif model_type == "LSTM":
        model.add(LSTM(64))
    elif model_type == "GRU":
        model.add(GRU(64))

    model.add(Dropout(0.3))
    model.add(Dense(1, activation="sigmoid"))

    model.compile(
        optimizer="adam",
        loss="binary_crossentropy",
        metrics=["accuracy"]
    )

    return model

results = {}
histories = {}
```

```
models = {}

for model_type in ["RNN", "LSTM", "GRU"]:
    print(f"\nTraining {model_type} model...")

    model = build_model(model_type)

    start = time.time()

    history = model.fit(
        X_train,
        y_train,
        epochs=3,
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )

    training_time = time.time() - start

    y_pred = (model.predict(X_test, verbose=0) > 0.5).astype(int).ravel()

    results[model_type] = {
        "Accuracy": accuracy_score(y_test, y_pred) * 100,
        "Precision": precision_score(y_test, y_pred) * 100,
        "Recall": recall_score(y_test, y_pred) * 100,
        "F1 Score": f1_score(y_test, y_pred) * 100,
        "Training Time": training_time
    }

    histories[model_type] = history
    models[model_type] = model

print("\nTraining completed!")

df = pd.DataFrame(results).T

print(df.round(2))

df[["Accuracy", "Precision", "Recall", "F1 Score"]].plot(
    kind="bar",
    figsize=(10, 5)
)

plt.title("RNN vs LSTM vs GRU - IMDB Sentiment Analysis")
plt.ylabel("Score (%)")
plt.xlabel("Model")
plt.xticks(rotation=0)
plt.grid(axis="y", alpha=0.3)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()

df["Training Time"].plot(
    kind="bar",
    color=["blue", "green", "orange"],
```

```
    figsize=(7, 4)
)

plt.title("Training Time Comparison")
plt.xlabel("Model")
plt.ylabel("Time (seconds)")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 5))

for name, history in histories.items():
    plt.plot(
        history.history["val_accuracy"],
        label=name
    )

plt.title("Validation Accuracy Comparison")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```